

Diseño de fragmentación horizontal

Ejercicio 4 · AdventureWorks2022 · Documento de análisis y procedimiento reproducible

Tablas: Purchasing.PurchaseOrderHeader (propietaria) y Purchasing.PurchaseOrderDetail (miembro).

Diseño: cinco nodos lógicos por rangos de PurchaseOrderID. **Esquema del experimento:** Prot_Frag.

Alcance de los resultados. No se dispone de una conexión a la instancia de AdventureWorks2022. Las estadísticas, fronteras, cardinalidades y verificaciones prácticas se deben registrar tras ejecutar

Tarea_diseno_fragmentacion.sql. No se presentan valores ni capturas simulados.

1. Selección y modelo relacional

Se descentraliza lógicamente el control de abastecimiento y compras mediante fragmentación horizontal primaria de las órdenes de compra y fragmentación horizontal derivada de sus partidas. La clave PurchaseOrderID vincula las cabeceras con los detalles en una relación 1:N.

Limitación geográfica: PurchaseOrderID no identifica el lugar real de origen. Los nodos 1 a 5 son convenciones lógicas; se necesitaría un atributo o un mapeo fiable de origen para asignación geográfica real.

2. Segmento del grafo relacional

Relación fuente (propietaria)	Relación destino (miembro)
Purchasing.PurchaseOrderHeader PK: PurchaseOrderID	Purchasing.PurchaseOrderDetail PK: (PurchaseOrderID, PurchaseOrderDetailID) FK: PurchaseOrderID → PurchaseOrderHeader

Arista dirigida: PurchaseOrderHeader → PurchaseOrderDetail. Vendor, Employee y ShipMethod son referencias de la cabecera, pero no se fragmentan de forma derivada en este ejercicio.

3. Estadísticas, límites y predicados simples

Primero se obtiene mínimo, máximo y número de órdenes, además del total de partidas. Después, **ntile(5)** ordena por PurchaseOrderID y reparte las filas en cinco grupos aproximadamente equilibrados. Los máximos de los grupos 1 a 4 producen $b1 < b2 < b3 < b4$.

```
select min(PurchaseOrderID), max(PurchaseOrderID), count_big(*)  
from Purchasing.PurchaseOrderHeader;
```

Predicados candidatos: $P = \{p1: ID \leq b1, p2: ID \leq b2, p3: ID \leq b3, p4: ID \leq b4\}$. Las condiciones $ID > bi$ son negaciones de estos predicados. El primer y último intervalo carecen de frontera exterior para permitir futuras inserciones; el equilibrio puede variar con el tiempo.

4. Aplicaciones y selección de predicados (COM_MIN)

Carga de trabajo **ilustrativa, no medida**: Q1...Q5 consultan por PurchaseOrderID uno de los cinco segmentos, con una frecuencia de referencia de 20 ejecuciones/día cada una. Las frecuencias deberán sustituirse por mediciones si el diseño se lleva a producción.

Consulta	Condición consultada	Frecuencia hipotética
Q1	$ID \leq b1$	20/día
Q2	$b1 < ID \leq b2$	20/día
Q3	$b2 < ID \leq b3$	20/día
Q4	$b3 < ID \leq b4$	20/día
Q5	$ID > b4$	20/día

Bajo esta carga, COM_MIN selecciona $P' = \{p1, p2, p3, p4\}$: cada umbral diferencia accesos a segmentos contiguos y omitir cualquiera fusionaría dos regiones que las consultas distinguen. La completitud y minimalidad son relativas a Q1...Q5; no se afirman para una carga desconocida.

5. Fragmentación primaria (PHORIZONTAL)

Aunque existen $2^4 = 16$ combinaciones teóricas de predicados, las implicaciones $p1 \Rightarrow p2 \Rightarrow p3 \Rightarrow p4$ permiten solo cinco minterms consistentes. Se define $H_i = \sigma_{mi}(\text{Purchasing.PurchaseOrderHeader})$.

Fragmento / nodo	Minterm simplificado	Rango para SQL
Prot_Frag.POH_1 / 1	$p1$	$ID \leq b1$
Prot_Frag.POH_2 / 2	$\neg p1 \wedge p2$	$b1 < ID \leq b2$
Prot_Frag.POH_3 / 3	$\neg p2 \wedge p3$	$b2 < ID \leq b3$
Prot_Frag.POH_4 / 4	$\neg p3 \wedge p4$	$b3 < ID \leq b4$
Prot_Frag.POH_5 / 5	$\neg p4$	$ID > b4$

El script crea las cinco copias con **select ... into**. La pertenencia definitiva se decide por rangos de ID, no por el número de grupo de ntile durante la consulta. El cuadro de límites se conserva en Prot_Frag.Limites.

6. Fragmentación derivada (SEMI-JOIN)

Para $i = 1...5$, cada D_i contiene las filas de Purchasing.PurchaseOrderDetail cuyo PurchaseOrderID existe en H_i . La condición **exists** evita duplicar partidas y conserva solo las columnas de detalle.

```
select d.* into Prot_Frag.POD_1
from Purchasing.PurchaseOrderDetail as d
where exists (
    select 1 from Prot_Frag.POH_1 as h
    where h.PurchaseOrderID = d.PurchaseOrderID
);
```

La completitud de los detalles presupone integridad referencial: cada partida debe referenciar exactamente una cabecera. El script repite este procedimiento para los cinco nodos.

7. Reconstrucción y reglas de corrección

Reconstrucción: se crean las vistas Prot_Frag.POH_Global y Prot_Frag.POD_Global como unión de sus cinco fragmentos mediante **union all**, para conservar la multiplicidad de las filas.

```
create or alter view Prot_Frag.POH_Global as
select * from Prot_Frag.POH_1
union all select * from Prot_Frag.POH_2
union all select * from Prot_Frag.POH_3
union all select * from Prot_Frag.POH_4
union all select * from Prot_Frag.POH_5;
```

Regla	Fundamento	Verificación de la consulta
Compleitud	Cada fila de cabecera pertenece a un rango; cada detalle con FK válida encuentra su cabecera.	Faltantes = 0 y Sobrantes = 0 (except en ambos sentidos).
Reconstrucción	La unión de los cinco fragmentos reproduce la relación original.	TotalOriginal = TotalReconstruido, además de no haber diferencias.
Disyunción	Los intervalos son excluyentes y cada clave de detalle se asigna una sola vez.	LlavesDuplicadas = 0 para cada vista.

Criterio para registrar “APROBADO” en cada relación: Faltantes = 0, Sobrantes = 0, TotalOriginal = TotalReconstruido y LlavesDuplicadas = 0. El script devuelve los indicadores, pero no declara aprobación automáticamente. No ejecutar el experimento mientras existan escrituras concurrentes sobre las tablas fuente.

8. Procedimiento de ejecución y evidencia

Paso 1. Abrir una instancia de SQL Server con AdventureWorks2022 y ejecutar el archivo

Tarea_diseno_fragmentacion.sql. Este selecciona la base de datos y crea Prot_Frag si no existe.

Paso 2. Guardar las estadísticas originales de cabeceras y detalles; no incorporar números supuestos.

Paso 3. Registrar el resultado de Prot_Frag.Limites y las cuatro fronteras b1...b4. Comprobar que existen cinco grupos no vacíos.

Paso 4. Verificar la creación de Prot_Frag.POH_1...POH_5 y Prot_Frag.POD_1...POD_5. Anotar las cardinalidades de cada par de fragmentos.

Paso 5. Consultar las vistas globales, guardar las dos filas de validación (POH y POD) y aplicar el criterio de aprobación anterior.

Paso 6. Incorporar capturas o tablas de resultados reales al informe, cuando se disponga de acceso a la instancia. Conservar el SQL que produjo las evidencias.

9. Registro de resultados de la ejecución

Completar las siguientes tablas **exclusivamente con valores devueltos por SQL Server**. Los espacios sin datos indican que la prueba práctica está pendiente.

Estadística	Valor obtenido
MinimoID / MaximoID	_____
TotalOrdenes	_____
TotalDetalles	_____

Grupo	MinID	MaxID	Filas
1	_____	_____	_____
2	_____	_____	_____
3	_____	_____	_____
4	_____	_____	_____
5	_____	_____	_____

Relación	Faltantes	Sobrantes	Original / reconstruido	Llaves duplicadas
POH	_____	_____	____ / ____	_____
POD	_____	_____	____ / ____	_____

Resultado de validación: POH: _____ POD: _____ (registrar “APROBADO” solo si todos los indicadores cumplen el criterio).

10. Conclusiones y consideraciones de implantación

El diseño produce cinco fragmentos primarios excluyentes por PurchaseOrderID y cinco fragmentos derivados con reglas explícitas de reconstrucción. El equilibrio aproximado se refiere al número de cabeceras, no necesariamente al volumen de detalles o a la frecuencia de consulta. La corrección formal depende de la clave no nula y de la integridad referencial; la verificación empírica continúa pendiente hasta ejecutar el script.

El experimento realiza copias locales mediante **select ... into**: no distribuye datos entre servidores y no copia claves primarias, foráneas, índices ni triggers. Una implementación real exige distribución física, restricciones, enrutamiento, sincronización, transacciones y recuperación.

11. Entregables y referencia bibliográfica

Entregables: este PDF actualizado y Tarea_diseno_fragmentacion.sql. En caso de requerir resultados ejecutados, adjuntar las evidencias reales obtenidas en los pasos 2 a 5.

Özsü, M. T. y Valdúriez, P. (2020). Principles of Distributed Database Systems, 4.^a ed., Springer. Cap. 2, secciones 2.1.1.1–2.1.1.4: grafo de joins, predicados, COM_MIN, PHORIZONTAL, fragmentación derivada y reglas de corrección. Material señalado en el documento original.